

Project Name: EVOLVE - Software Change Request

Authors: Alan V Jacob & Josna Jose

Subject: Advanced Software Evolution

1. Software Requirements Specification (SRS)

1.1 Introduction and Purpose

The system, named EVOLVE, is a Software Change Request Management System. The primary purpose of EVOLVE is to provide a web-based application specifically designed to manage the comprehensive lifecycle of software change requests (CRs). It is built to assist engineering teams in tracking these requests, analyzing the resulting ripple effects of proposed changes across the software codebase, and actively managing the workflow involved in implementing those changes.

1.2 Overall System Description

EVOLVE operates as a reactive Single Page Application (SPA). This interface allows both developers and engineering managers to submit new change requests, visualize the current state of all requests on an interactive Kanban board, perform detailed analyses of the risks and impacts associated with proposed changes, and maintain traceability by viewing code-level modifications.

1.3 Functional Requirements

The core capabilities of the system are defined by the following functional requirements:

- **FR-1:** Users must possess the ability to create a new Change Request (CR) by specifying a Title, Description, Priority (categorized as Low, Medium, High, Critical), Category (Feature, Bug Fix, Refactoring, Optimization), and the specific Affected Components.
- **FR-2:** The application must automatically generate a unique ID for every newly created CR.
- **FR-3:** New CRs must be assigned a default status of "Draft" upon creation.
- **FR-4:** The system is required to display all CRs on a Kanban board utilizing predefined columns: Draft, Submitted, Impact Analysis, Approved, In Progress, Testing, and Merged.

- **FR-5:** Users must be able to seamlessly change the status of any CR by dragging and dropping the respective CR card between the Kanban columns.
- **FR-6:** The impact analysis module must calculate an evolution risk score ranging from 0 to 100, alongside a risk level designation (Low, Medium, High, Critical) for any components affected by a given CR.
- **FR-7:** The risk and impact calculation must accurately propagate through the component dependencies. This calculation assesses transitive impacts by factoring in the distance, complexity, and churn rate of the components.
- **FR-8:** The system must visually map and display the change impact propagation, illustrating both direct and transitive impacts.
- **FR-9:** The system must generate simulated, synthetic code diffs corresponding to the components affected by a CR.
- **FR-10:** Users must have the ability to view these generated code diffs to ensure proper traceability.

1.4 Non-Functional Requirements

To ensure optimal performance and maintainability, the system must adhere to the following constraints:

- **NFR-1:** The User Interface (UI) must be highly responsive, updating reactively to internal state changes without requiring full page reloads.
- **NFR-2:** The UI must be constructed using a modern design system, incorporating a specific font stack and utilizing CSS variables for consistent styling.
- **NFR-3:** Impact analysis calculations are required to execute in real-time, specifically under 100ms for typical codebase sizes, to ensure the UI remains fluid and uninterrupted.
- **NFR-4:** As a strict architectural constraint, the application must be developed utilizing Vanilla JavaScript and CSS. Heavy front-end frameworks, such as React or Angular, are strictly prohibited.

2. System Design Document

2.1 Architecture Overview and Tech Stack

EVOLVE is architected exclusively as a modular, frontend-only Single Page Application (SPA). By leveraging a custom component-based architecture alongside a centralized, reactive state management pattern, the system completely removes the dependency on external frontend frameworks like React or Vue.

- **Core Technologies:** HTML5, Vanilla JavaScript utilizing ES Modules, and CSS3.

- **Build Tool:** Vite.
- **External Libraries:** Chart.js is utilized exclusively for analytics and data visualization purposes.

2.2 High-Level Component Design

The application is logically separated into discrete modules: the core application shell, state management, algorithmic utilities, data mocks, and UI components. Each UI component, housed within the `/src/components/` directory, is structured as an ES6 class that accepts a DOM container and implements `init()` and `render()` lifecycle methods.

- **Dashboard.js:** Serves as the main landing view, rendering key metrics and charts.
- **Kanban.js:** Implements the drag-and-drop board responsible for managing the lifecycle statuses of CRs.
- **ImpactAnalysis.js:** Responsible for visualizing the dependency graph and displaying calculated risk scores for a specifically selected CR.
- **Traceability.js:** Renders the inline code diffs for components affected by system changes.
- **Header.js:** Manages global navigation and triggers global actions, such as initiating the "Create CR" modal.

2.3 State Management

State is centralized using a **StateStore** within `src/utils/store.js`, implementing a robust Publish-Subscribe (Pub/Sub) pattern.

- **State Tree:** Maintains the overarching state including `activeView`, `changeRequests`, `codebase metadata`, and selection contexts like `selectedComponentId` and `activeImpactCR`.
- **Reactivity:** UI components act as subscribers to the store. Whenever the state mutates through designated actions (such as `addChangeRequest` or `updateCRStatus`), the store notifies all subscribers, which subsequently triggers the necessary UI re-renders.

2.4 Application Data Flow

The sequence of operations during user interaction follows a strict unidirectional flow:

1. **User Action:** The user interacts directly with the UI, such as dropping a CR card into a different Kanban column.

2. **State Update:** The respective UI component invokes a store action, for example, `store.updateCRStatus(id, newStatus)`.
3. **Mutation:** The centralized store processes the action and updates its internal state tree.
4. **Notification:** The store executes the `notify()` function, which systematically invokes all registered subscriber callbacks.
5. **Re-render:** The primary App controller, located in `main.js`, receives the state change notification and proceeds to re-render the active component utilizing the freshly updated data.

2.5 Impact Engine Architecture

Located in `src/utils/impactEngine.js`, the Impact Engine houses the core business logic required for change propagation.

- **Graph Traversal:** The engine utilizes a Breadth-First Search (BFS) algorithm traversing an inverted dependency graph (propagation graph) to determine impact pathways.
- **Risk Calculation:** The algorithm formulates an evolution risk score by calculating the traversal distance (differentiating between direct and transitive impacts), individual component complexity, and historical churn rate.

2.6 UI/UX Design System

The visual presentation of EVOLVE is controlled via a global design system.

- **Styling:** A coherent visual theme dictating colors, spacing, and layouts is maintained globally via CSS variables housed within `variables.css`.
- **Typography:** The application integrates a modern web font stack consisting of Outfit, JetBrains Mono, and Plus Jakarta Sans to establish a clear text hierarchy.

3. Implementation Details

3.1 Project Structure

The project leverages Vite as a build tool and is organized into a highly modular directory structure:

- `/src/main.js`: Acts as the primary application entry point. It is responsible for bootstrapping the app, handling routing between different views, and managing the global CR creation modal.
- `/src/components/`: Contains all UI component classes, including Dashboard, Kanban, ImpactAnalysis, and Traceability.

- `/src/utils/`: Houses the core logic utilities, specifically `store.js` for state management and `impactEngine.js` for graph traversal calculations.
- `/src/data/`: Stores the initial seed data files, namely `initialCRs.js` and `mockCodebase.js`, which are utilized for system demonstration.
- `/src/styles/`: Contains all modular CSS stylesheets, separated into `variables.css`, `main.css`, and `components.css`.

3.2 Key Implementation Patterns

- **Vanilla SPA Routing:** Routing is handled dynamically without a router library. The App class within `main.js` listens closely to store changes. It dynamically swaps out the active UI component (switching between the Dashboard, Kanban, or Impact views) by directly manipulating the innerHTML of the main application container.
- **Drag and Drop API:** The Kanban component (`Kanban.js`) directly implements the native HTML5 Drag and Drop API. It utilizes standard `dragstart`, `dragover`, and `drop` events to successfully capture CR ID payloads and update the corresponding state statuses seamlessly.
- **Impact Algorithm Logic:** The `impactEngine.js` reverses the standard codebase dependencies. Instead of mapping "what relies on who," it traces "who relies on what" to accurately simulate change propagation. The BFS queue-based traversal ensures both direct and transitive impacts are mapped efficiently.

4. Testing Strategy and Documentation

Because the system relies entirely on vanilla JavaScript, the testing strategy dictates that business logic must be covered in strict isolation, followed by integration tests intended for the UI layer.

4.1 Unit Testing

Unit tests are targeted at the core utilities, with Vitest or Jest recommended as the primary testing tools.

State Store (`store.js`) Testing:

- Tests must ensure that the `addChangeRequest` function properly instantiates a new CR, complete with a generated ID, and accurately appends the dummy diffs.
- Tests must verify that `updateCRStatus` successfully alters the status of the precisely targeted CR without causing mutations to other existing records.
- The Pub/Sub mechanism must be tested to guarantee that all active subscribers are accurately called whenever the global state undergoes a change.

Impact Engine (impactEngine.js) Testing:

- Tests require mocking a small, isolated component graph (e.g., $A \rightarrow B \rightarrow C$).
- An assertion must be written to ensure that passing component A as a seed correctly flags component B as experiencing a direct impact, and component C as experiencing a transitive impact.
- Tests must strictly assert that the calculated risk scores accurately apply the designated decay formula, accounting for distance, complexity, and churn rate parameters.

4.2 Integration Testing

Integration testing focuses on DOM interactions, utilizing tools such as Testing Library (DOM) or Cypress.

- **Component Rendering:** Tests must assure that whenever store.activeView is updated, the App shell correctly mounts the corresponding UI component to the DOM.
- **Form Submission:** Tests must simulate a user filling out the CR Modal inside main.js. The test must ensure that completing the submission successfully triggers store.addChangeRequest and that the Kanban board subsequently reflects the newly added item.

4.3 Manual and End-to-End Testing (UAT)

A standardized set of manual User Acceptance Testing (UAT) cases must be verified to ensure complete system functionality:

1. **CR Creation:** The tester must click "Create CR", populate the form data, select multiple affected components, and submit the request. The verification step requires confirming the new CR appears correctly in the "Draft" column of the Kanban board.
2. **Lifecycle Movement:** The tester must manually drag a CR card from the "Draft" column and drop it into "In Progress". The verification step requires confirming the card visually anchors in the new column and that its internal status state is accurately updated within the store.
3. **Impact Simulation:** The tester must click the "Impact" button located on a CR card. The verification step requires ensuring the view immediately switches to the Impact Analysis screen, displaying the correct risk scores and affected components derived from the initial CR creation parameters.

4. **Code Traceability:** The tester must click the "Diff" button on a CR card. The verification step confirms that the application view switches to the Traceability interface and properly displays the generated synthetic code diffs.